TRIBALSETU

AI-Enabled Scholarship & Fellowship Management System

Final Technical & Product Blueprint

A single source of truth for the 6-member, 7-day MVP: product scope, architecture, frontend, backend, database, authentication, authorization, AI/OCR, workflow, security, scalability, testing, deployment, observability, and engineering rules.

Ministry of Tribal Affairs • Smart Education • SIH MVP

Implementation principle

Design for production-quality foundations from the first line of code, but implement only the smallest vertical slice required for a convincing 7-day MVP.

1. Executive Summary

TribalSetu is an end-to-end digital platform for administering scholarship and fellowship schemes for the Ministry of Tribal Affairs. The system addresses manual scrutiny, repeated correspondence, multiple verification stages, delays, limited visibility, and workflow/verification errors.

The MVP centers on one complete lifecycle: scheme configuration → application → document upload → OCR/extraction → rule-based eligibility → human verification → deficiency/resubmission → eligibility → selection → notification → audit trail.

The system is intentionally modular so additional schemes, rules, fields, document requirements and selection criteria can be configured rather than hard-coded.

Locked technology decisions

Area	Decision
Product	TribalSetu
Frontend	Next.js + TypeScript + Tailwind CSS + shadcn/ui
Client data/API	TanStack Query
Forms/validation	React Hook Form + Zod
Backend	Python + FastAPI + Pydantic
ORM/migrations	SQLAlchemy 2.0 + Alembic
Database	Supabase PostgreSQL
File storage	Supabase Storage, private buckets
Authentication	JWT + Argon2/bcrypt password hashing
Authorization	RBAC + resource checks + workflow checks
MVP roles	Applicant / Reviewer / Admin
Architecture	Next.js → REST → FastAPI → PostgreSQL/Storage

2. Functional Scope

Applicant

- Register and authenticate.
- Browse available schemes and view scheme details.
- Create, edit and save application drafts.
- Complete dynamic application forms.
- Upload required documents and view upload/verification status.
- Submit an application.
- Track status and workflow timeline.
- Receive and respond to deficiencies.
- Resubmit corrected information/documents.
- View eligibility and selection outcomes and in-app notifications.

Reviewer

- Authenticate and access assigned applications only.
- View application details and submitted information.
- View documents through authorized signed URLs.
- View OCR/extracted values and document-to-application comparisons.
- View explainable eligibility evaluations.
- Verify information and record remarks.
- Raise deficiencies.
- Recommend eligible/ineligible within permitted workflow states.

Admin

- Manage users and roles.
- Create/manage schemes and scheme versions.
- Configure application fields.
- Configure required documents.
- Configure eligibility rules.
- Assign reviewers.
- Monitor applications and system statistics.
- Manage/record selection outcomes.
- Review audit logs.

Explicitly out of MVP scope

- Mobile application, Kubernetes, microservices, MinIO/S3, complex ML training, live government integrations, Aadhaar integration, payments, SMS infrastructure, advanced IAM, real-time chat, chatbot, and a sophisticated workflow designer.
- Do not add major features after Day 6.

3. System Architecture

Primary rule: Next.js is the presentation/client layer. FastAPI is the authoritative business backend. The frontend does not directly perform business operations against PostgreSQL.

Logical architecture

Applicant / Reviewer / Admin → Next.js → REST/JSON → FastAPI → SQLAlchemy → Supabase PostgreSQL; FastAPI also authorizes document access and interacts with Supabase Storage for private files.

Backend responsibility boundaries

Concern	Owner
UI, navigation, rendering	Next.js
Client-side form UX validation	React Hook Form + Zod
Server-side validation	FastAPI/Pydantic
Authentication	FastAPI
Authorization	FastAPI
Business rules	FastAPI services
Workflow transitions	FastAPI workflow service
Eligibility evaluation	FastAPI rule engine
OCR/extraction orchestration	Python/FastAPI
Persistent data	PostgreSQL
Document bytes	Supabase Storage
Audit trail	FastAPI + PostgreSQL

Recommended service layering

Router → Service → Repository/DB. Routers translate HTTP requests. Services own business logic and orchestration. Repositories isolate database operations where useful. SQLAlchemy models should not contain large business workflows.

4. Frontend Blueprint

Route structure

app/(auth)/login; app/applicant/dashboard; app/applicant/schemes; app/applicant/applications; app/applicant/applications/[id]; app/applicant/notifications; app/reviewer/dashboard; app/reviewer/applications; app/reviewer/applications/[id]; app/admin/dashboard; app/admin/users; app/admin/schemes; app/admin/schemes/[id]; app/admin/selection; app/admin/audit.

Frontend folder structure

frontend/app • components/ui • components/layout • components/forms • components/applications • components/documents • features/auth • features/applications • features/schemes • features/documents • features/eligibility • features/review • features/admin • lib/api.ts • lib/auth.ts • hooks • types

Frontend engineering rules

- Use TypeScript strictly; avoid any unless justified.
- Centralize API calls in a typed API layer; components should not contain raw fetch logic everywhere.
- Use TanStack Query for server state, caching, loading/error states and invalidation.
- Use React Hook Form + Zod for complex forms and client-side validation.
- Treat client validation as UX only; the backend is the security/validation boundary.
- Create reusable components for status badges, document cards, timelines, tables, modals and form fields.
- Keep role-specific layouts separate while sharing primitives.
- Handle loading, empty, error and success states on every major screen.
- Do not hard-code business eligibility decisions into UI components.

5. Backend Blueprint

Backend structure

backend/app/main.py; core/config.py; core/database.py; core/security.py; auth/; users/; roles/; schemes/; applications/; documents/; eligibility/; review/; workflow/; selection/; notifications/; audit/; alembic/; tests/.

Per-domain module

router.py → service.py → repository.py → models.py → schemas.py

Backend engineering rules

- Use SQLAlchemy 2.0 style and AsyncSession with asyncpg.
- Use Alembic for all schema changes; never manually edit production tables as a normal workflow.
- Use UUID primary keys; also provide human-readable application_number such as NFST-2026-000142.
- Use Pydantic schemas to define request/response contracts.
- Centralize authorization policies/dependencies rather than repeating ad hoc checks.
- Use transactions for multi-step state-changing operations.
- Keep secrets in environment variables; never commit them.
- Return stable, documented error shapes and meaningful HTTP status codes.
- Add request/correlation IDs to important requests so logs and audit entries can be connected.

Initial API surface

POST /api/v1/auth/register; POST /api/v1/auth/login; GET /api/v1/auth/me; GET/POST/PATCH scheme endpoints; POST/GET/PATCH application endpoints; POST /applications/{id}/documents; POST /applications/{id}/submit; POST /applications/{id}/transition; reviewer application/review/deficiency endpoints; admin user/scheme/configuration/audit endpoints.

6. Database & Data Model

Core tables

users, roles, permissions, user_roles, role_permissions, applicant_profiles, schemes, scheme_versions, form_fields, eligibility_rules, document_requirements, applications, application_data, application_documents, application_assignments, reviews, deficiencies, eligibility_evaluations, status_history, selection_results, notifications, audit_logs.

Key relationships

User → ApplicantProfile; User ↔ Roles ↔ Permissions; Scheme → SchemeVersion → FormFields/EligibilityRules/DocumentRequirements; SchemeVersion → Applications; Application → Documents/Assignments/Reviews/Deficiencies/EligibilityEvaluations/StatusHistory/SelectionResult.

Data modeling rules

- User is the authentication identity; ApplicantProfile is the scholarship participant profile.
- Every application belongs to exactly one scheme version.
- Eligibility evaluates against the scheme version attached to the application.
- Use relational columns for stable/common attributes and JSONB for configurable application-specific fields.
- Do not create separate NFSTApplication and NOSApplication tables for every scheme variation.
- Preserve historical scheme versions instead of mutating rules under existing applications.
- Store document metadata in PostgreSQL and actual files in private Storage.
- Use created_at/updated_at consistently; use explicit timestamps for state transitions and decisions.

Five non-negotiable invariants

- Every application belongs to exactly one scheme version.
- Eligibility is evaluated against that scheme version.
- Status changes only through valid workflow transitions.
- Every protected action passes server-side authorization.
- Important decisions and configuration changes create audit records.

7. Authentication, Authorization & Security

Authentication flow

Register → hash password → store user → login → verify password → issue JWT → client sends Bearer token → FastAPI validates token → load active user → determine roles → authorize request.

Authentication rules

- Never store plaintext passwords.
- Use Argon2 or bcrypt for password hashing.
- JWTs must have expiry and should contain only minimal claims.
- Never trust a role supplied by the frontend.
- New registrations default to APPLICANT; elevated roles are assigned by authorized administration.
- Plan for refresh-token/session hardening if the project later moves beyond the MVP.

Authorization model

Authorization = authenticated user + role permission + resource access + valid workflow state.

Role	Representative permissions
Applicant	Create/read/update own application, submit, upload own documents, respond to deficiencies
Reviewer	Read assigned applications, read assigned documents, review, raise deficiency, recommend eligibility
Admin	Manage users, schemes, rules, documents, assignments, selection, audit

Security baseline

- HTTPS in deployed environments.
- Strict CORS configuration; do not use wildcard origins in production.
- Validate file type, size and upload context; never trust client-provided MIME type alone.
- Private storage buckets; use short-lived signed URLs after authorization.
- Prevent IDOR by checking ownership/assignment server-side for every protected resource.
- Rate-limit authentication endpoints in production.
- Do not log passwords, JWTs, API keys, or unnecessary sensitive PII.
- Use parameterized ORM queries; never concatenate user input into SQL.
- Apply least privilege to service/database credentials.
- Maintain secure headers and input/output validation at the API boundary.

8. Workflow & State Machine

DRAFT → SUBMITTED → UNDER_VERIFICATION → (DEFICIENCY_RAISED → RESUBMITTED → UNDER_VERIFICATION) or ELIGIBLE/INELIGIBLE → UNDER_SELECTION → SELECTED/NOT_SELECTED → POST_SELECTION.

Allowed actors

Transition	Actor
DRAFT → SUBMITTED	Applicant
SUBMITTED → UNDER_VERIFICATION	System/authorized officer
UNDER_VERIFICATION → DEFICIENCY_RAISED	Reviewer
DEFICIENCY_RAISED → RESUBMITTED	Applicant
UNDER_VERIFICATION → ELIGIBLE/INELIGIBLE	Reviewer/authorized officer
ELIGIBLE → UNDER_SELECTION	Authorized selection role/admin capability
UNDER_SELECTION → SELECTED/NOT_SELECTED	Authorized selection authority
SELECTED → POST_SELECTION	Authorized officer/system

Implementation rule

Never expose arbitrary PATCH status mutation. Use a transition service/endpoint that validates current state, requested transition, actor permission, ownership/assignment and required conditions. The transition should atomically update status, write status history, write audit information and trigger relevant notification work.

Eligible is not the same as Selected

Eligibility establishes whether the applicant satisfies configured requirements. Selection is a separate stage with its own authority, criteria and audit trail.

9. AI / OCR / Document Intelligence

Pipeline

Upload → private Storage → OCR → extraction → normalized structured values → compare with application data → rule evaluation → explainable result → reviewer.

AI design principles

- AI assists reviewers; it does not silently make the final administrative decision.
- Keep deterministic eligibility rules separate from probabilistic AI observations.
- Persist enough evidence for a reviewer to understand why a check passed, failed or needs attention.
- Represent document mismatches as review signals rather than automatic final rejection.
- Use configurable rules instead of hard-coded NFST/NOS conditions.
- For the 7-day MVP, keep OCR/extraction inside the Python/FastAPI ecosystem unless workload forces separation.

Eligibility operators for MVP

EQUALS, NOT_EQUALS, GREATER_THAN, GREATER_THAN_OR_EQUAL, LESS_THAN, LESS_THAN_OR_EQUAL, IN, NOT_IN, EXISTS, NOT_EXISTS.

Explainable evaluation record

rule_id + actual_value + expected_value + result + reason + evaluated_at. Do not store only eligible=true/false.

Example

Application: Category = ST; certificate OCR: Category = ST; configured rule: category IN [ST]; result: PASS; reason: extracted category matches configured requirement.

10. Document Storage & Privacy

Supabase Storage is the MVP file store. Use a private bucket. PostgreSQL stores metadata and the relationship between a document and an application.

Document metadata

id, application_id, document_requirement_id, file_name, storage_path, file_type, file_size, uploaded_at, verification_status and any required processing metadata.

Secure access flow

User → FastAPI authentication → ownership/assignment check → generate short-lived signed URL → client views/downloads document.

Operational rules

- Do not expose sensitive document buckets publicly.
- Use generated storage paths rather than trusting arbitrary user-supplied paths.
- Limit file sizes and allowed document types.
- Consider virus/malware scanning before production deployment.
- Keep document access auditable for sensitive environments.
- Do not duplicate files unnecessarily.

11. Scalability & Production Readiness

The MVP should be simple, but the foundations should avoid dead ends. The architecture is intentionally stateless at the API layer so multiple FastAPI instances can be added later.

Scalability from day one

- Use stateless JWT-authenticated API requests where practical.
- Use connection pooling and sane database connection limits.
- Paginate application, audit and notification lists.
- Add database indexes on foreign keys, status, scheme_version_id, applicant_id, reviewer assignment and timestamps.
- Avoid N+1 ORM queries; use eager loading/select-in loading deliberately.
- Use asynchronous processing for OCR and other slow work when the workload requires it.
- Keep API instances horizontally scalable; avoid storing authoritative session state in process memory.
- Use caching only where measured or clearly beneficial; do not introduce Redis solely for architecture diagrams.
- Keep files outside PostgreSQL; Storage scales independently from relational metadata.
- Use versioned APIs such as /api/v1.

Future evolution

- Background job queue for OCR/AI processing.
- Dedicated AI/document-processing worker service if throughput grows.
- Redis for distributed caching/queues if justified.
- Object-storage abstraction if migration away from Supabase becomes necessary.
- Read replicas/advanced database tuning at higher scale.
- Centralized observability and managed secret management.

What not to do now

Do not build microservices, Kubernetes or a distributed event bus during the 7-day MVP. A well-structured modular FastAPI monolith is easier to finish, test and demonstrate.

12. API Contract & Error Design

The frontend and backend team must work from one contract document. Every endpoint should specify method, path, authentication requirement, permission, request schema, response schema, errors and side effects.

Example contract

POST /api/v1/applications Auth: required Permission: APPLICATION_CREATE Input: scheme_version_id + application data Output: application id + application_number + status Errors: invalid scheme version, forbidden, validation error

Recommended error shape

{ code, message, details, request_id } — stable machine-readable code for frontend handling; human-readable message for users/developers; details only where safe; request_id for support/debugging.

API principles

- Use consistent plural resources and versioned paths.
- Use correct HTTP status codes.
- Do not leak internal stack traces or database errors.
- Return only fields the caller is authorized to see.
- Paginate large lists.
- Use idempotency or duplicate-submission protection where repeated actions could cause damage.

13. Testing Strategy

Backend tests

- Unit tests for password hashing/auth utilities, rule engine and workflow transitions.
- Authorization tests: applicant cannot access another applicant's application; reviewer cannot access unassigned application.
- Integration tests for critical application submission and review flows.
- Database migration test from clean database.

Frontend tests

- Form validation and important user interactions.
- Role-based route protection and empty/loading/error states.
- Critical application journey smoke test.

End-to-end golden path

Applicant login → select scheme → create draft → upload document → submit → reviewer sees assignment → reviewer raises deficiency → applicant resubmits → reviewer marks eligible → selection recorded → applicant sees result → audit verified.

Security test checklist

- Invalid/expired JWT rejected.
- Applicant ownership enforcement works.
- Reviewer assignment enforcement works.
- Admin-only endpoints reject other roles.
- Invalid workflow transitions rejected.
- Unauthorized document access rejected.
- Oversized/disallowed uploads rejected.
- Sensitive data absent from logs.

14. Logging, Observability & Deployment

Logging

- Use structured logs with timestamp, level, request_id, user_id when appropriate, endpoint and outcome.
- Never log passwords, access tokens, secret keys or unnecessary document contents.
- Separate developer/debug logging from production-safe logging.

Metrics to care about

- API request latency and error rate.
- Authentication failures.
- Application submissions.
- Document processing failures.
- OCR processing time.
- Eligibility evaluation failures.
- Queue/backlog if background processing is introduced.
- Database connection saturation.

Deployment baseline

- Separate local/development and production environment variables.
- Use HTTPS in deployment.
- Run database migrations as a controlled deployment step.
- Build frontend and backend reproducibly.
- Keep secrets in the hosting provider's secret/environment configuration.
- Use a health endpoint for backend availability.
- Seed only safe demo data; never use real applicant data in the demo.

CI/CD

- On pull request: lint, type-check, unit tests and backend tests.
- On deploy: build, migrate, smoke test.
- Protect the main branch and require review for shared/core changes where practical.

15. Team & Ownership

Member	Primary ownership
1 — Backend/Architecture	Database, SQLAlchemy, Alembic, Auth, JWT, RBAC, applications, workflow, admin backend, integration
2 — Applicant Frontend	Login, applicant dashboard, schemes, forms, documents, tracking, deficiency response
3 — Reviewer Frontend	Reviewer dashboard, queue, application detail, document/OCR/eligibility views, review actions
4 — AI/OCR	OCR, extraction, document comparison, AI assistance, rule engine, explainability
5 — Selection/Analytics	Selection workflow, scoring, selection dashboard, statistics/reports; assist integration
6 — DevOps/Integration/QA	Supabase, deployment, environment, seed data, integration tests, bug tracking, demo environment

Shared engineering rules

- All cross-team contracts live in /docs.
- No frontend developer invents an API response without updating the contract.
- No backend developer changes response shape without coordinating with frontend.
- No direct database access from the frontend.
- Every important feature must respect RBAC and workflow state.
- Use small pull requests and descriptive commit messages.
- Protect shared schema/core files with team communication.

16. Seven-Day Build Plan

Day	Primary outcome
Day 1	Project setup, Supabase, SQLAlchemy, Alembic, users/roles/permissions, JWT, /me, frontend shell/login
Day 2	ApplicantProfile, schemes, versions, form fields, applications, documents, create/submit flow
Day 3	State machine, assignments, reviewer queue/detail, review, deficiencies, resubmission, status history
Day 4	OCR, extraction, document comparison, AI assistance, eligibility engine
Day 5	Selection, admin configuration, dashboards, audit, notifications
Day 6	Full integration, validation, loading/error states, responsive polish, bug fixing
Day 7	Feature freeze, final testing, deployment, demo data, presentation and rehearsal

Milestones

- End Day 1: login + JWT + /me + role permissions work.
- End Day 2: applicant can create and submit an application; reviewer can see assigned application.
- End Day 3: deficiency/resubmission lifecycle works.
- End Day 4: document intelligence produces a visible, explainable result.
- End Day 5: selection + audit + admin view works.
- End Day 6: golden path works end-to-end.
- End Day 7: stable demo, no new major features.

17. Golden Demo Script

1. Applicant

Login → choose NFST → fill application → upload certificate → submit.

2. Intelligence

OCR extracts document fields → system compares them with application data → rule engine produces explainable checks.

3. Reviewer

Reviewer logs in → sees assigned application → opens documents → sees OCR/AI evidence → reviews.

4. Deficiency

Reviewer raises a deficiency → applicant is notified → applicant corrects/uploads → resubmits.

5. Eligibility & selection

Reviewer verifies → application becomes eligible → selection stage → selected/not selected → applicant sees result.

6. Admin & audit

Admin sees application statistics, scheme configuration and audit events.

Target demo duration: 5–7 minutes. The demo should emphasize the problem solved, explainable intelligence, human oversight, transparent workflow and auditability.

18. Definition of Done

A feature is done only when:

- The UI exists and handles loading/error/empty/success states.
- The backend endpoint exists with Pydantic request/response schemas.
- Authentication and authorization are enforced server-side.
- The database migration exists.
- The feature respects application workflow states.
- Important state-changing actions create audit/history records.
- The feature is tested at the appropriate unit/integration/E2E level.
- API contract documentation is updated.
- No secrets or sensitive data are committed/logged.
- The feature works in the deployed demo environment.

Release gate

- No critical authorization bypasses.
- No broken golden-path workflow.
- No public sensitive document bucket.
- No unhandled database migration mismatch.
- No major console/runtime errors in the demo.
- Demo data is synthetic and reproducible.

19. The First-Line-of-Code Checklist

Before writing business features, create the project with the following foundations already planned:

Order	Foundation
1	Repository structure and README
2	Environment/config management
3	FastAPI app + health endpoint
4	Supabase PostgreSQL connection
5	SQLAlchemy 2.0 AsyncSession
6	Alembic migrations
7	User/Role/Permission models
8	Password hashing + JWT
9	Current-user dependency
10	Permission/RBAC dependency
11	Audit/request ID foundation
12	Initial tests
13	API contract document
14	Only then: schemes/applications/workflow

The principle

Do not build authentication, security, scalability or audit as a last-minute patch. Build the seams for them at the beginning, while keeping the implementation simple enough to finish in seven days.

20. Final Locked Blueprint

TRIBALSETU — AI-Enabled Scholarship & Fellowship Management System

Dimension	Locked decision
Product goal	End-to-end scholarship/fellowship administration
MVP users	Applicant, Reviewer, Admin
Frontend	Next.js + TypeScript + Tailwind + shadcn/ui
Client state	TanStack Query
Forms	React Hook Form + Zod
Backend	FastAPI + Pydantic
ORM	SQLAlchemy 2.0 Async
Migrations	Alembic
Database	Supabase PostgreSQL
Files	Private Supabase Storage
Auth	JWT + Argon2/bcrypt
Authorization	RBAC + resource + workflow
AI	OCR/extraction + assistive verification
Eligibility	Configurable deterministic rule engine
Workflow	Controlled state transitions
Audit	Status history + audit logs
Scalability	Stateless API, pagination, indexes, async processing path
Security	Server-side authorization, private files, validation, least privilege, safe logs
Testing	Unit + integration + golden-path E2E
Deployment	Environment-based config, migrations, HTTPS, CI/CD, health checks
MVP strategy	One complete vertical slice, not every possible feature

North Star: A reviewer should be able to understand an application's data, documents, AI/OCR evidence, eligibility checks, deficiencies, decisions and history from one coherent workflow—while the applicant can clearly see what is required, what happened, what needs correction and what the current status is.

This document is the baseline to follow from the first line of code.